



K.R. Mangalam University

School of Engineering and Technology

Program: BCA (AI & DS)

Course Code: ENCA351

Course: Design and Analysis of Algorithms

LAB 2 – Algorithm Performance Measurement and Benchmarking Tool

Submitted By: Akash Sharma

Roll Number: 2401201108

Section: B

1. Introduction

The objective of this lab is to develop a Python-based tool for measuring and comparing the performance of different algorithms.

The project measures:

- Execution time
- Memory usage
- Number of comparisons/operations
- Performance for different input sizes

The implemented algorithms include Linear Search, Binary Search, Single Loop, Nested Loop, Recursive Factorial, and Iterative Factorial.

2. Question 1 — Project Setup

The project workspace was created using Python, a virtual environment, required libraries, Git, and GitHub.

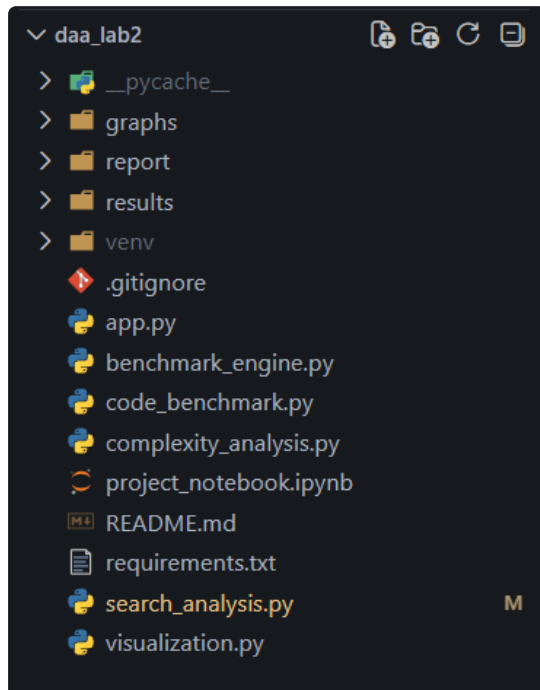
Technologies Used

Technology	Purpose
Python	Programming language
Streamlit	Interactive web application
Pandas	Data processing
NumPy	Numerical operations
Matplotlib	Graph generation
Tracemalloc	Memory measurement
Jupyter Notebook	Experimental analysis
Git/GitHub	Version control

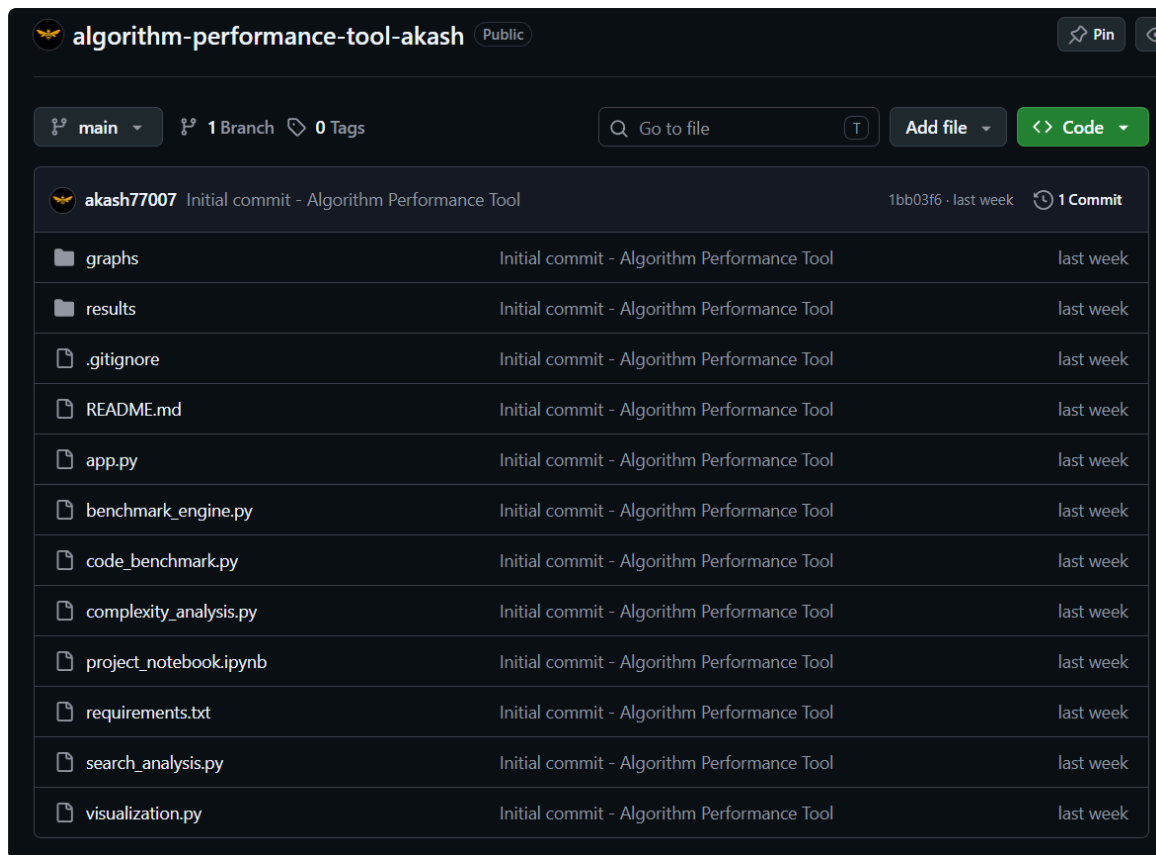
Project Structure

```
daa_lab2/
|
├─ app.py
├─ search_analysis.py
├─ code_benchmark.py
├─ benchmark_engine.py
├─ complexity_analysis.py
├─ visualization.py
├─ project_notebook.ipynb
├─ requirements.txt
├─ README.md
|
├─ graphs/
│   ├── search_time.png
│   ├── factorial_time.png
│   ├── loop_time.png
│   └── memory_comparison.png
|
└─ results/
    └─ benchmark_results.csv
```

Screenshot — Project Setup



Screenshot — GitHub Repository



3. Question 2 — Linear Search and Binary Search

Linear Search checks elements sequentially until the target is found.

Binary Search repeatedly divides a sorted dataset into two halves to locate the target.

Linear Search

Code

```
def linear_search(arr, target):
    comparisons = 0
    for i, value in enumerate(arr):
        comparisons += 1
        if value == target:
            return i, comparisons
    return -1, comparisons
```

Binary Search

Code

```
def binary_search(arr, target):  
  
    low = 0  
    high = len(arr) - 1  
    result = -1  
    comparisons = 0  
  
    while low <= high:  
        mid = (low + high) // 2  
        comparisons += 1  
        if arr[mid] == target:  
            result = mid  
            high = mid - 1  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return result, comparisons
```

Complexity

Algorithm	Best Case	Average Case	Worst Case	Space
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)

Search Output

Linear Search

Index

500

Comparisons

501

Binary Search

Index

500

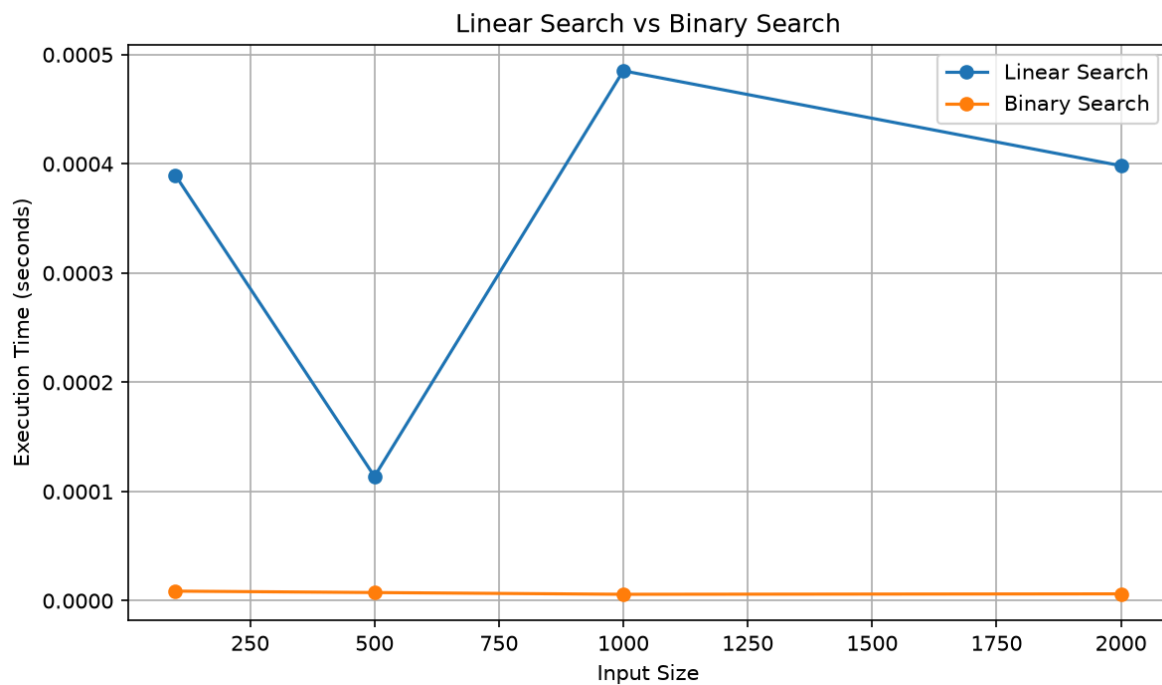
Comparisons

9

Performance Comparison

Better Algorithm: Binary Search

Search Performance



4. Question 3 — Code Benchmarking Module

The Code Benchmarking Module was developed to execute and compare the following programs:

1. Single Loop Traversal
2. Nested Loop Traversal
3. Recursive Factorial
4. Iterative Factorial

Code

```
# CODE BENCHMARKING MODULE

def single_loop(n):
    total = 0
    for i in range(n):
        total += i
    return total

def nested_loop(n):
    total = 0
    for i in range(n):
        for j in range(n):
            total += 1
    return total

def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

def factorial_iterative(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# TESTING

if __name__ == "__main__":

    print("Single Loop:")
    print(single_loop(10))

    print("\nNested Loop:")
    print(nested_loop(10))

    print("\nRecursive Factorial:")
    print(factorial_recursive(5))

    print("\nIterative Factorial:")
    print(factorial_iterative(5))
```

Complexity

Program	Time Complexity	Space Complexity
Single Loop	$O(n)$	$O(1)$
Nested Loop	$O(n^2)$	$O(1)$
Recursive Factorial	$O(n)$	$O(n)$
Iterative Factorial	$O(n)$	$O(1)$

Benchmark Output

Benchmark executed successfully!

Performance Result

Result: 4950

Execution Time: 1.469999915570952e-05 seconds

Single Loop:
45

Nested Loop:
100

Recursive Factorial:
120

Iterative Factorial:
120

5. Question 4 — Automated Benchmarking Engine

An automated benchmarking engine was implemented to execute algorithms for different input sizes and record their performance.

The engine records:

- Algorithm name

- Input size
- Execution time
- Memory usage

Code

```
class BenchmarkEngine:
    def __init__(self):
        self.results = []

    def run(self, func, input_sizes, name=None, arg_builder=lambda n: (n,)):
        algorithm_name = name or func.__name__

        for n in input_sizes:
            args = arg_builder(n)

            tracemalloc.start()
            start_time = time.perf_counter()
            func(*args)
            execution_time = time.perf_counter() - start_time
            current, peak_memory = tracemalloc.get_traced_memory()
            tracemalloc.stop()
            self.results.append({
                "Algorithm": algorithm_name,
                "Input Size": n,
                "Execution Time (seconds)": execution_time,
                "Memory (KB)": peak_memory / 1024
            })
        return self

    def as_dataframe(self):
        return pd.DataFrame(self.results)
```

The collected results are stored in:

```
results/benchmark_results.csv
```

Benchmark Results

```
(venv) D:\Documents\Assignments\daa_lab2>type results\benchmark_results.csv
Algorithm,Input Size,Execution Time (seconds),Memory (KB)
Linear Search,100,0.00038950000089243986,0.0703125
Linear Search,500,0.00011380000069038942,0.1328125
Linear Search,1000,0.00048519999836571515,0.09375
Linear Search,2000,0.000398300002416363,0.09375
Binary Search,100,8.799997885944322e-06,0.0234375
Binary Search,500,7.499998901039362e-06,0.0546875
Binary Search,1000,5.900001269765198e-06,0.0234375
Binary Search,2000,6.300000677583739e-06,0.0234375
Single Loop,100,2.3399999918183312e-05,0.0234375
Single Loop,500,5.8999998145736754e-05,0.0234375
Single Loop,1000,0.00015219999841065146,0.0234375
Single Loop,2000,0.00016829999731271528,0.0234375
Nested Loop,100,0.0010640999971656129,0.0546875
Nested Loop,500,0.02448029999868595,0.0234375
Nested Loop,1000,0.1501148999996076,0.0234375
Nested Loop,2000,0.38220860000001267,0.0234375
Recursive Factorial,100,0.00014279999959398992,0.1953125
Recursive Factorial,500,0.002229900001111673,5.26171875
Recursive Factorial,900,0.009319299999333452,17.82421875
Iterative Factorial,100,0.00011139999696752056,0.1953125
Iterative Factorial,500,0.000562299999728566,1.05859375
Iterative Factorial,900,0.0011082999990321696,2.04296875
```

6. Question 5 — Complexity Comparison

The theoretical complexity of all implemented algorithms was compared using the following table.

Algorithm	Best Case	Average Case	Worst Case	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Single Loop	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Nested Loop	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Recursive Factorial	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Iterative Factorial	$O(n)$	$O(n)$	$O(n)$	$O(1)$

Complexity Analysis Code

```
# COMPLEXITY ANALYSIS
```

```
complexity_data = [  
    {  
        "Algorithm": "Linear Search",  
        "Best Case": "O(1)",  
        "Average Case": "O(n)",  
        "Worst Case": "O(n)",  
        "Space": "O(1)"  
    },  
    {  
        "Algorithm": "Binary Search",  
        "Best Case": "O(1)",  
        "Average Case": "O(log n)",  
        "Worst Case": "O(log n)",  
        "Space": "O(1)"  
    },  
    {  
        "Algorithm": "Single Loop",  
        "Best Case": "O(n)",  
        "Average Case": "O(n)",  
        "Worst Case": "O(n)",  
        "Space": "O(1)"  
    },  
    {  
        "Algorithm": "Nested Loop",  
        "Best Case": "O(n²)",  
        "Average Case": "O(n²)",  
        "Worst Case": "O(n²)",  
        "Space": "O(1)"  
    },  
    {  
        "Algorithm": "Recursive Factorial",  
        "Best Case": "O(n)",  
        "Average Case": "O(n)",  
        "Worst Case": "O(n)",  
        "Space": "O(n)"  
    },  
    {  
        "Algorithm": "Iterative Factorial",  
        "Best Case": "O(n)",  
        "Average Case": "O(n)",  
        "Worst Case": "O(n)",  
        "Space": "O(1)"  
    }  
]
```

```
def display_complexity_table():

    print("\nComplexity Comparison")
    print("=" * 90)
    print(
        f"{'Algorithm':<25}"
        f"{'Best':<15}"
        f"{'Average':<15}"
        f"{'Worst':<15}"
        f"{'Space':<15}"
    )
    print("-" * 90)
    for item in complexity_data:
        print(
            f"{item['Algorithm']:<25}"
            f"{item['Best Case']:<15}"
            f"{item['Average Case']:<15}"
            f"{item['Worst Case']:<15}"
            f"{item['Space']:<15}"
        )
    )
if __name__ == "__main__":
    display_complexity_table()
```

Observed Performance

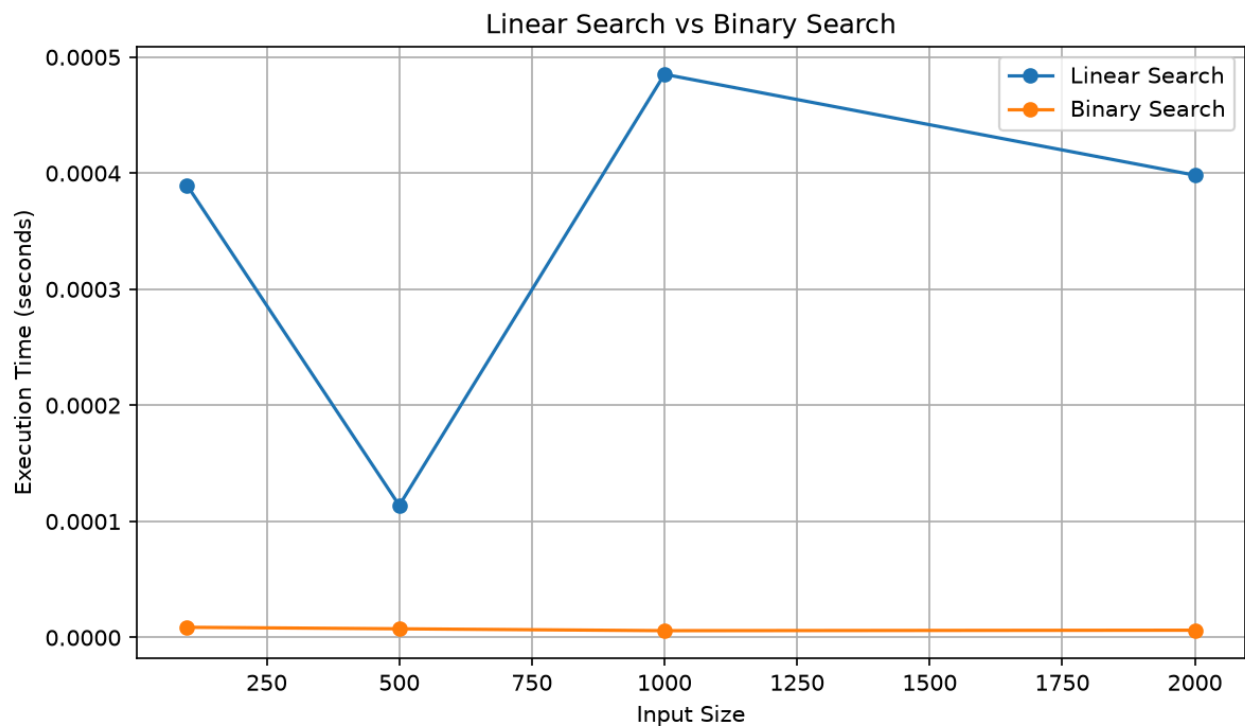
Algorithm / Pattern	Time Complexity	Space Complexity	Observed Growth
Linear Search	$O(n)$	$O(1)$	Linear growth
Binary Search	$O(\log n)$	$O(1)$	Very slow growth
Single Loop	$O(n)$	$O(1)$	Linear growth
Nested Loop	$O(n^2)$	$O(1)$	Quadratic growth
Recursive Factorial	$O(n)$	$O(n)$	Linear time, higher memory
Iterative Factorial	$O(n)$	$O(1)$	Linear growth, constant auxiliary space

7. Question 6 — Visualizations and Performance Observations

Performance graphs were generated using Matplotlib to compare the implemented algorithms.

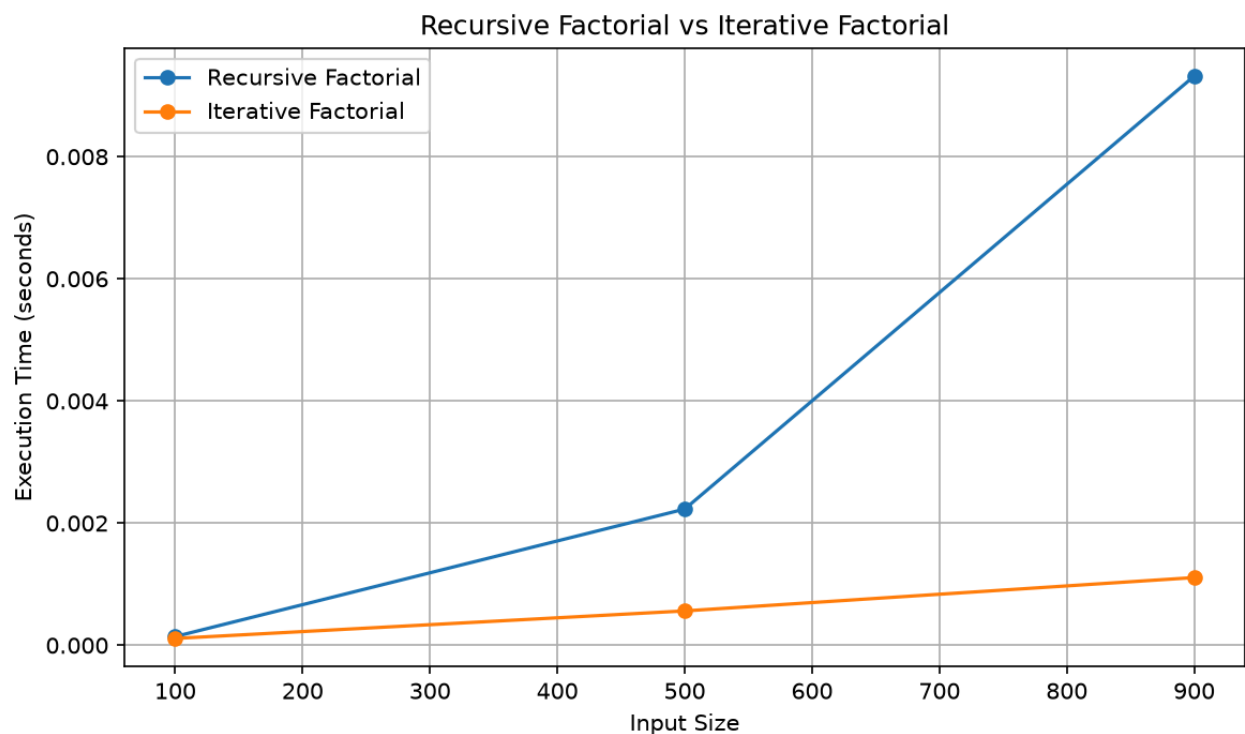
7.1 Search Algorithm Comparison

The graph compares the execution time of Linear Search and Binary Search for different input sizes.



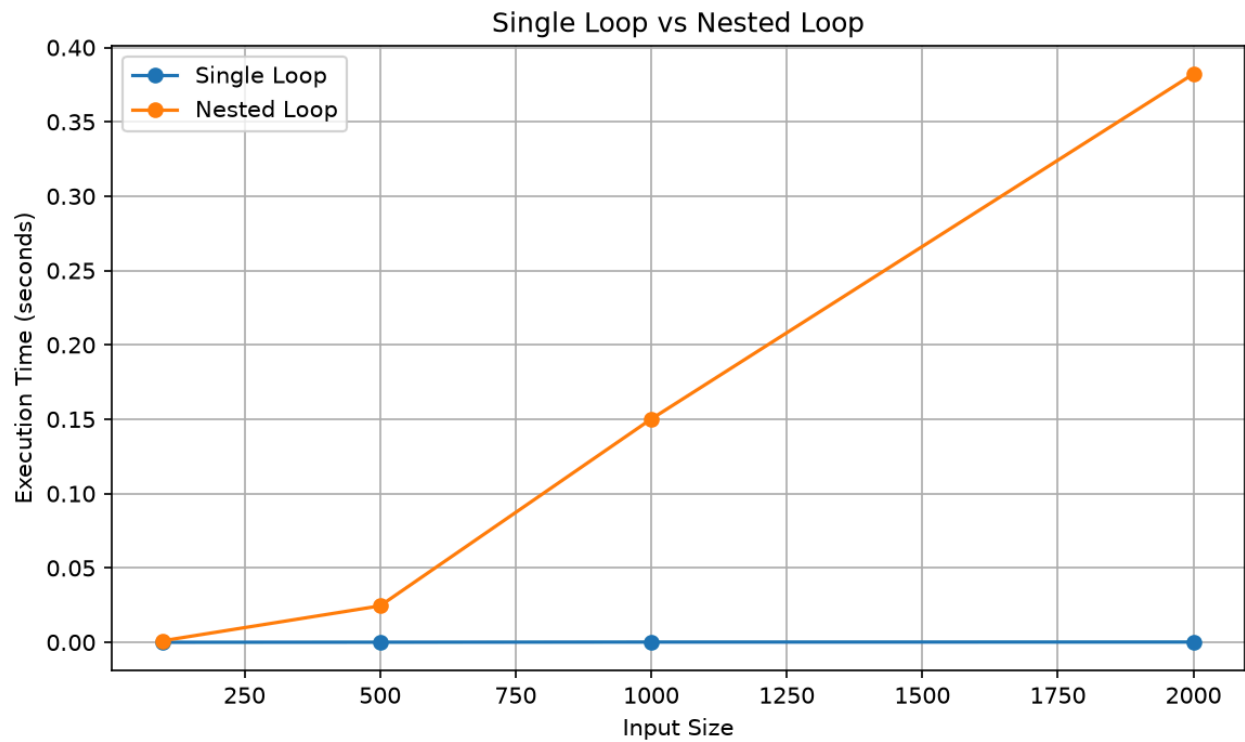
7.2 Factorial Comparison

The graph compares Recursive Factorial and Iterative Factorial.



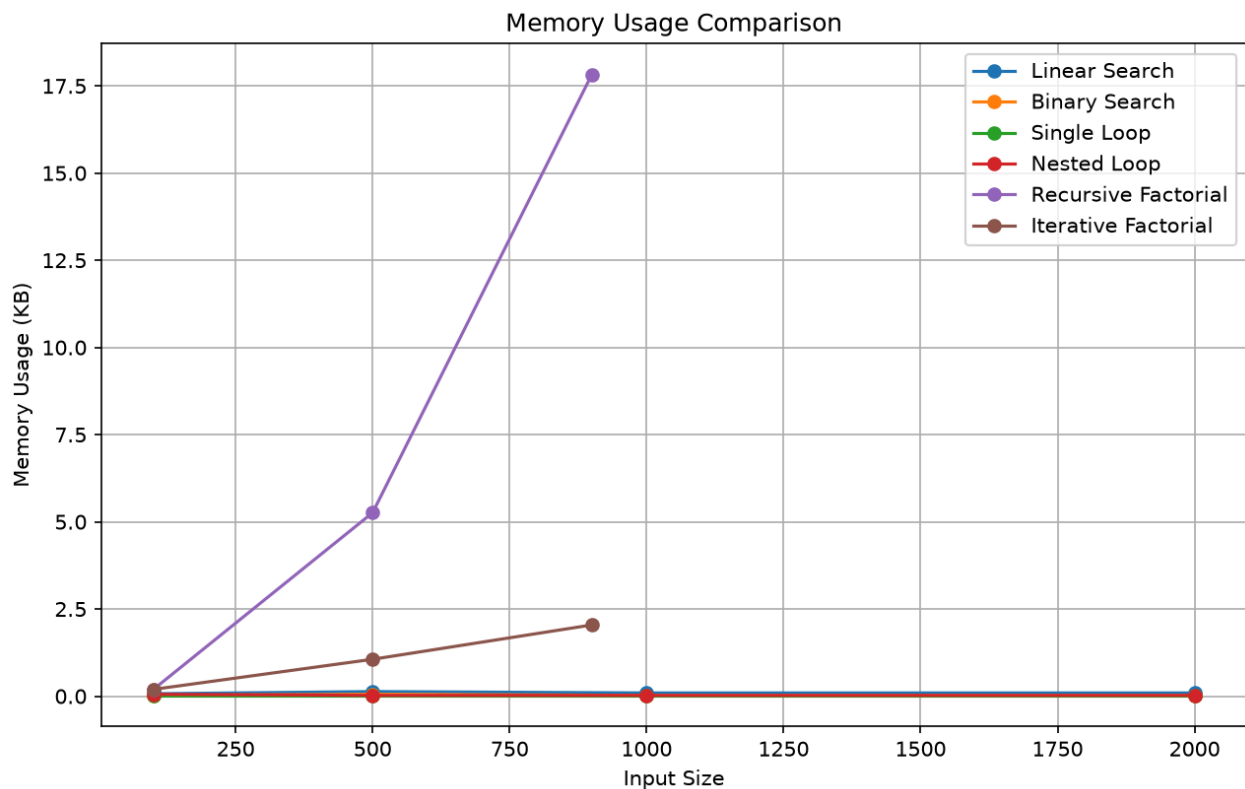
7.3 Loop Comparison

The graph compares Single Loop and Nested Loop performance.



7.4 Memory Comparison

The graph compares the memory usage of the implemented algorithms.



Performance Observations

- Linear Search checks elements sequentially.
 - Binary Search reduces the search space by half at every step.
 - Single Loop has linear growth with input size.
 - Nested Loop has quadratic growth and becomes significantly slower for larger inputs.
 - Recursive Factorial uses additional call-stack memory.
 - Iterative Factorial uses constant auxiliary space.
-

8. Question 7 — Benchmarking Report and Analysis

The complete benchmarking analysis combines theoretical complexity with experimental measurements.

Search Analysis

Linear Search performs sequential comparisons, while Binary Search repeatedly divides the sorted search space.

```
(venv) D:\Documents\Assignments\daa_lab2>python search_analysis.py
Linear Search:
Index: 1
Comparisons: 2

Binary Search:
Index: 1
Comparisons: 3
```

Factorial Analysis

Both Recursive and Iterative Factorial have $O(n)$ time complexity. However, the recursive implementation requires additional stack space, while the iterative implementation uses constant auxiliary space.

```
(venv) D:\Documents\Assignments\daa_lab2>python code_benchmark.py
Single Loop:
45

Nested Loop:
100

Recursive Factorial:
120

Iterative Factorial:
120
```

Loop Analysis

Single Loop performs a linear number of operations, while Nested Loop performs approximately n^2 operations.

Single Loop Traversal

Input Size

100

Selected Program: Single Loop Traversal

Input Size: 100

Run Benchmark

Benchmark executed successfully!

Performance Result

Result: 4950

Execution Time: 1.3799999578623101e-05 seconds

Nested Loop Traversal

Input Size

100

Selected Program: Nested Loop Traversal

Input Size: 100

Run Benchmark

Benchmark executed successfully!

Performance Result

Result: 10000

Execution Time: 0.0004961000013281591 seconds

Time-Space Trade-Off

The project demonstrates that algorithm selection depends on both time and space requirements.

Binary Search provides logarithmic search time but requires sorted data.

Recursive Factorial and Iterative Factorial have the same asymptotic time complexity but differ in auxiliary space usage.

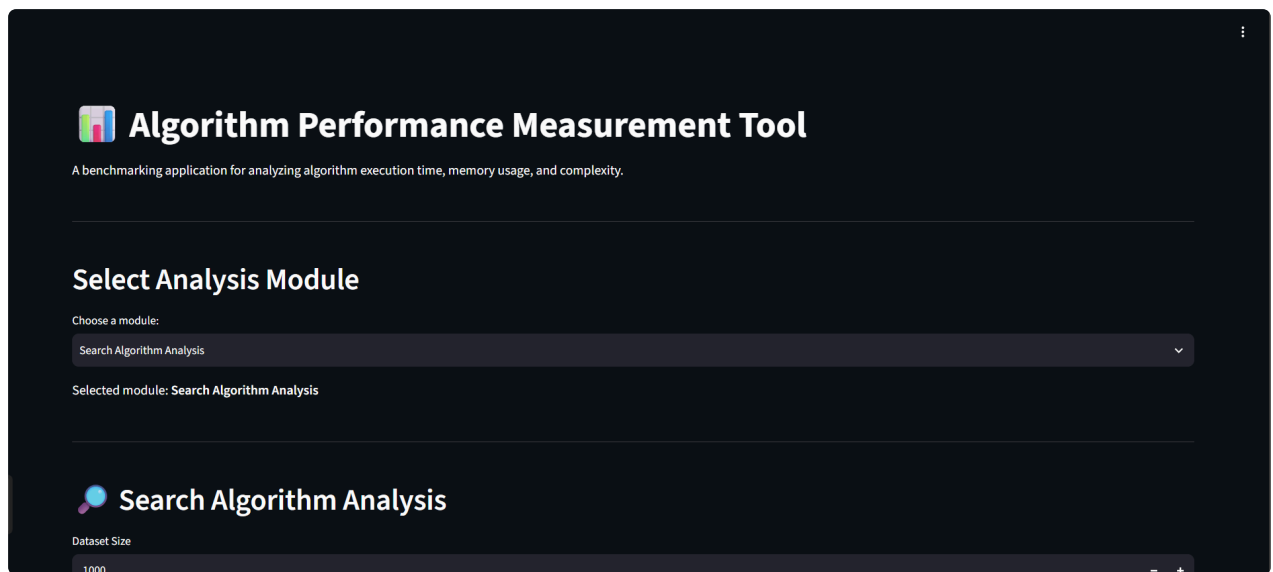
9. Streamlit Application

A Streamlit application was developed to provide an interactive interface for the performance measurement tool.

The application allows users to:

- Select an analysis module
- Enter input sizes
- Run algorithms
- View results
- Compare performance

Application Screenshot



10. Conclusion

The Algorithm Performance Measurement and Benchmarking Tool demonstrates the relationship between theoretical algorithmic complexity and practical performance.

The project combines:

- Algorithm implementation
- Performance measurement
- Memory analysis
- Complexity analysis
- Automated benchmarking
- Data collection
- Visualization
- Streamlit interface

The experiments demonstrate how increasing input size affects algorithm execution time and memory usage.

12. Repository

The complete project source code, graphs, benchmark results, notebook, and documentation are maintained in the GitHub repository.

| [akash77007/algorithm-performance-tool-akash](https://github.com/akash77007/algorithm-performance-tool-akash)